

四位全加器实验报告

2023010302 石亦烜

实验时间：2025年4月25日 座号：1

一、实验目的

1. 学习元件例化的方式进行硬件电路设计，了解结构化语言描述和行为式语言描述的区别
2. 学会利用软件仿真实现对数字电路的逻辑功能进行验证和分析

二、实验内容

1. 先设计一位全加器，然后通过元件实例化的方式实现串行进位的四位全加器
2. 设计超前进位的四位全加器
3. 使用 SystemVerilog 自带的加法运算实现四位全加器，并将结果采用十进制表示，并使用自带译码的七段数码管显示结果

三、实验步骤和代码

1. 一位全加器和串行进位四位全加器设计 源代码full_adder.sv如下：

```
`timescale 1ns / 1ps
// 指定时间单位和精度，这里表示时间单位为1ns，仿真时的精度为1ps

//一位全加器模块
module one_full_adder(
    input wire a, b, cin,      // 输入,两个加数和进位
    output wire sum, cout     // 输出加法结果和进位
);
    assign sum = a ^ b ^ cin; // 根据真值表化简后的表达式，计算和
    assign cout = (a & b) | (cin & (a ^ b)); // 计算进位
endmodule

module four_full_adder(
    input wire [3:0]A,B,
    input wire Cin,
    output wire [3:0]F,
    output wire Cout
);
    wire [2:0]C; //存储串行进位过程中的中间进位
    //实例化
    one_full_adder F0(.a(A[0]),.b(B[0]),.cin(Cin),.sum(F[0]),.cout(C[0]));
    one_full_adder F1(.a(A[1]),.b(B[1]),.cin(C[0]),.sum(F[1]),.cout(C[1]));
    one_full_adder F2(.a(A[2]),.b(B[2]),.cin(C[1]),.sum(F[2]),.cout(C[2]));
    one_full_adder F3(.a(A[3]),.b(B[3]),.cin(C[2]),.sum(F[3]),.cout(Cout));
endmodule
```

2. 超前进位四位全加器设计 源代码parallel_adder.sv如下：

```

`timescale 1ns / 1ps

//并行加法器模块
module parallel_adder(
    input wire [3:0]A,B,
    input wire Cin,
    output wire [3:0]F,
    output wire Cout
);
    wire [3:0]G,P,C;
    assign G=A&B;           //产生进位
    assign P=A^B;           //传递进位
    assign C[0]=Cin;
    assign C[1]=G[0]|(P[0]&Cin);
    assign C[2]=G[1]|(P[1]&G[0])|(P[1]&P[0]&Cin);
    assign C[3]=G[2]|(P[2]&G[1])|(P[2]&P[1]&G[0])|(P[2]&P[1]&P[0]&Cin);
    assign F=P^C;
    assign Cout=G[3]|(P[3]&G[2])|(P[3]&P[2]&G[1])|(P[3]&P[2]&P[1]&G[0])|
(P[3]&P[2]&P[1]&P[0]&Cin);
endmodule

```

上述两实验的约束文件均为four_full_adder.xdc，代码如下：

```

# A input
set_property -dict {PACKAGE_PIN M21 IOSTANDARD LVCMOS33} [get_ports A[3]];
#I01接插孔
set_property -dict {PACKAGE_PIN N20 IOSTANDARD LVCMOS33} [get_ports A[2]];
#I02接插孔
set_property -dict {PACKAGE_PIN N22 IOSTANDARD LVCMOS33} [get_ports A[1]];
#I03接插孔
set_property -dict {PACKAGE_PIN P21 IOSTANDARD LVCMOS33} [get_ports A[0]];
#I04接插孔

# B input
set_property -dict {PACKAGE_PIN T21 IOSTANDARD LVCMOS33} [get_ports B[3]];
#I06接插孔
set_property -dict {PACKAGE_PIN U21 IOSTANDARD LVCMOS33} [get_ports B[2]];
#I07接插孔
set_property -dict {PACKAGE_PIN R21 IOSTANDARD LVCMOS33} [get_ports B[1]];
#I08接插孔
set_property -dict {PACKAGE_PIN R22 IOSTANDARD LVCMOS33} [get_ports B[0]];
#I09接插孔

# Cin input
set_property -dict {PACKAGE_PIN P22 IOSTANDARD LVCMOS33} [get_ports Cin];      #I05
接插孔

# F output
set_property -dict {PACKAGE_PIN AB18 IOSTANDARD LVCMOS33} [get_ports F[3]];

```

```

#I017接插孔
set_property -dict {PACKAGE_PIN AA20 IOSTANDARD LVCMOS33} [get_ports F[2]];
#I018接插孔
set_property -dict {PACKAGE_PIN AB21 IOSTANDARD LVCMOS33} [get_ports F[1]];
#I019接插孔
set_property -dict {PACKAGE_PIN AA21 IOSTANDARD LVCMOS33} [get_ports F[0]];
#I020接插孔

# Cout output
set_property -dict {PACKAGE_PIN AA18 IOSTANDARD LVCMOS33} [get_ports Cout];
#I016接插孔

set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]

```

3. 使用 SystemVerilog 自带的加法运算实现四位全加器，并将结果采用十进制表示，并使用自带译码的七段数码管显示结果 源代码adder.sv如下：

```

`timescale 1ns / 1ps

module adder(
    input wire [3:0]A,B,
    input wire Cin,
    output reg [3:0]F0, //七段数码管显示十进制数的个位
    output reg [1:0]F1 //七段数码管显示的十进制数的十位，由于四位全加器最大结果为31，所以只需要2位
);
    //使用verilog自带的加法运算实现加法器，结果用十进制在自带译码七段数码管上显示
    wire [4:0]temp_F;
    assign temp_F=A+B+Cin;
    always_comb begin
        F1 = temp_F / 10; // 计算十位
        F0 = temp_F % 10; // 计算个位
    end
endmodule

```

约束文件adder.xdc如下：

```

# A input
set_property -dict {PACKAGE_PIN M21 IOSTANDARD LVCMOS33} [get_ports A[3]];
#I01接插孔
set_property -dict {PACKAGE_PIN N20 IOSTANDARD LVCMOS33} [get_ports A[2]];
#I02接插孔
set_property -dict {PACKAGE_PIN N22 IOSTANDARD LVCMOS33} [get_ports A[1]];
#I03接插孔
set_property -dict {PACKAGE_PIN P21 IOSTANDARD LVCMOS33} [get_ports A[0]];
#I04接插孔

```

```

# B input
set_property -dict {PACKAGE_PIN T21 IOSTANDARD LVCMOS33} [get_ports B[3]];
#I06接插孔
set_property -dict {PACKAGE_PIN U21 IOSTANDARD LVCMOS33} [get_ports B[2]];
#I07接插孔
set_property -dict {PACKAGE_PIN R21 IOSTANDARD LVCMOS33} [get_ports B[1]];
#I08接插孔
set_property -dict {PACKAGE_PIN R22 IOSTANDARD LVCMOS33} [get_ports B[0]];
#I09接插孔

# Cin input
set_property -dict {PACKAGE_PIN P22 IOSTANDARD LVCMOS33} [get_ports Cin];      #I05
接插孔

# F0 output
set_property -dict {PACKAGE_PIN AB18 IOSTANDARD LVCMOS33} [get_ports F0[3]];
#I017接插孔
set_property -dict {PACKAGE_PIN AA20 IOSTANDARD LVCMOS33} [get_ports F0[2]];
#I018接插孔
set_property -dict {PACKAGE_PIN AB21 IOSTANDARD LVCMOS33} [get_ports F0[1]];
#I019接插孔
set_property -dict {PACKAGE_PIN AA21 IOSTANDARD LVCMOS33} [get_ports F0[0]];
#I020接插孔

# F1 output
set_property -dict {PACKAGE_PIN AB22 IOSTANDARD LVCMOS33} [get_ports F1[1]];
#I015接插孔
set_property -dict {PACKAGE_PIN AA18 IOSTANDARD LVCMOS33} [get_ports F1[0]];
#I016接插孔

set_property CFGBVS VCC0 [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]

```

四、仿真结果

1. 仿真文件testbench.sv如下:

```

`timescale 1ns / 1ps

//仿真模块
module testbench();
    reg [3:0]A,B;
    reg Cin;
    wire [3:0]F;
    wire Cout;
    wire [1:0]F1;      //十进制输出的十位
    wire [3:0]F0;      //十进制输出的个位
    integer i,j,k;     //用于遍历所有输入可能

    //实例化各个加法器模块
    //实例化串行进位全加器加法器模块

```

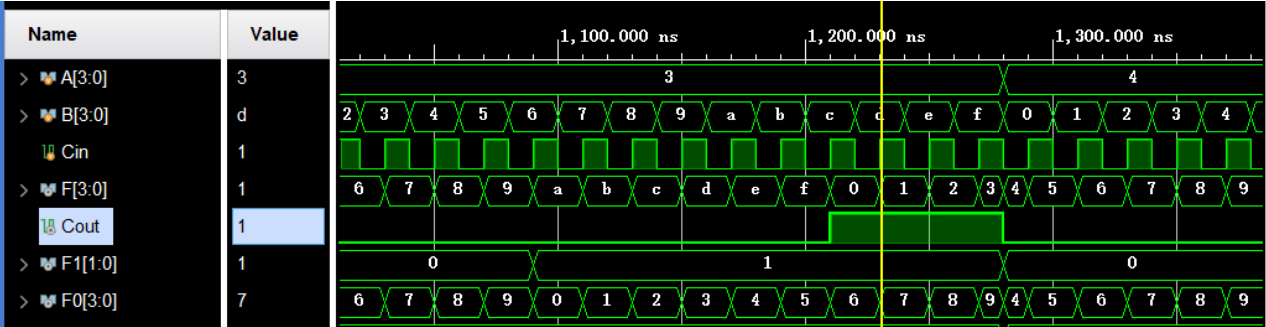
```
four_full_adder first_adder(
    .A(A),
    .B(B),
    .Cin(Cin),
    .F(F),
    .Cout(Cout)
);

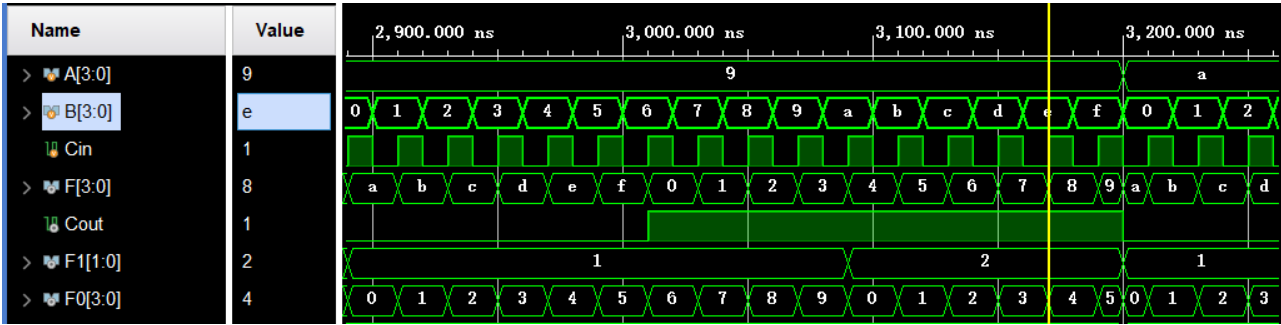
//实例化超前进位加法器模块
parallel_adder second_adder(
    .A(A),
    .B(B),
    .Cin(Cin),
    .F(F),
    .Cout(Cout)
);

//实例化十进制输出的加法器模块
adder third_adder(
    .A(A),
    .B(B),
    .Cin(Cin),
    .F0(F0),
    .F1(F1)
);

initial begin
    //遍历所有可能的输入组合
    for (i = 0; i < 16; i = i + 1) begin
        for (j = 0; j < 16; j = j + 1) begin
            for (k = 0; k < 2; k = k + 1) begin
                A = i;
                B = j;
                Cin = k;
                #10; // 等待10个时间单位
            end
        end
    end
    $finish; //结束仿真
end
endmodule
```

2. 仿真结果： 由于遍历了所有可能的输入组合，仿真结果较多， 以下仅展示部分结果：





可以看到，串行进位和超前进位的四位全加器的输出结果是相同的，且与十进制输出的结果一致。经过实际电路烧写验证，结果与仿真结果一致。

五、实验总结

本次实验过程中遇到了一些问题，主要是对 SystemVerilog 语言的语法含义不熟悉，导致在编写代码时出现了一些错误。经过查阅资料和请教老师，最终解决了这些问题。问题如下：

`assign` 和 `always` 语句的使用：在开始的代码中我定义了 `wire` 信号并在 `always_comb` 块中赋值，导致发生了错误。后面我明白在 SystemVerilog 中，`wire` 需要用 `assign` 语句进行连续赋值，是信号一变化立刻就会响应，所以实际是并行执行的，不能在 `always` 块中赋值，这也是 `wire` 变量和 C 语言中变量的一大不同。

通过本次实验，我对 FPGA 的实验环境和流程有了更深入的了解，也掌握了使用 Vivado 进行硬件设计和仿真的基本方法。同时，我也认识到在实际电路设计中，元件实例化是一种非常重要的设计方法，可以提高代码的可读性和可维护性。